



SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Investigation of Convolutional Neural Networks for Iterative Solver Selection

Leon Unterberger





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Investigation of Convolutional Neural
Networks for Iterative Solver Selection**

**Untersuchung von Convolutional Neural
Networks zur Auswahl iterativer Löser**

Author:	Leon Unterberger
Examiner:	Bungartz, Hans-Joachim
Supervisor:	Liu Weng, Hayden
Submission Date:	23.07.2026

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 23.07.2026

Leon Unterberger

Declaration of AI Usage

The following AI tools were used during the preparation of this thesis:

- **Claude Code (Claude Sonnet 4.6, Anthropic, accessed 2025–2026)** was used throughout the development of the experimental pipeline. This includes assistance with writing, debugging, and refactoring Python scripts for data generation, model training, and evaluation, as well as help with Docker configuration and L^AT_EX formatting. All generated code was reviewed, tested, and in many parts written or substantially modified by me.
- **Claude (Claude Sonnet 4.6, Anthropic, accessed 2025–2026)** was used for structural guidance during thesis writing, such as organising sections and suggesting what content belongs in each chapter. It was also used for proofreading drafts and identifying unclear passages. All text in the final thesis was written or rewritten by me; no AI-generated text was included verbatim.

AI tools were not used to generate experimental results, to interpret data on my behalf, or to produce any figures. The scientific content, the analysis of results, and all conclusions are entirely my own work.

Munich, 23.07.2026

Leon Unterberger

Acknowledgments

Special thanks to the very fantastic and outstanding princess verena strobl, which as my girlfriend, did her best to always distract me from work, begged me for food when I was focused and took care that i really needed the 4 months of working time. Thanks my little gnome.

Abstract

Solving large sparse linear systems $Ax = b$ requires to select a fitting iterative solver and preconditioner. Choosing one manually requires deep mathematical knowledge about solvers and their best applications. To automate the selection without professional mathematical insights, the usage of machine learning is a natural choice. The MM-Autosolver paper already proposes an interesting usage of a machine learning approach where a CNN is combined with an MLP approach to combine structural and numerical features to predict pairs of preconditioners and solvers. This yields an improvement of 2.94% when it is compared to a run without the CNN channel, so only with the MLP approach. Within this thesis the impact of 14 different downsampling methods for the CNN and the usage of additional features are evaluated. Furthermore, experiments like adding another CNN channel to combine different downsampling methods, ensembling multiple trained models and introducing a non-convergence penalty are implemented, which all in all increased the importance of structural features to about 8% in the best cases.

Contents

Acknowledgments	iv
Abstract	v
Abbreviations	1
1 Introduction	2
1.1 Motivation	2
1.2 Research Gap and Problem Statement	2
1.3 Research Goals and Contributions	3
1.4 Thesis Structure	4
2 Related Work	5
2.1 Iterative Solvers and Preconditioners	5
2.2 Automated Solver Selection	5
2.3 MM-AutoSolver	5
2.4 The Supervisor’s Solver-Evaluation Framework	5
2.5 Image Representations of Sparse Matrices	5
2.6 Convolutional Neural Networks for Classification	5
3 Methodology	6
3.1 Problem Formulation	6
3.2 Dataset Construction	6
3.2.1 Synthetic Data Generation	6
3.2.2 SuiteSparse Benchmark Matrices	9
3.2.3 Dataset Statistics	9
3.3 Feature Engineering	11
3.3.1 MM-AutoSolver Reference Features	11
3.3.2 Original 14 Features	12
3.3.3 Extended 20-Feature Set	12
3.4 Image Representations	14
3.5 Model Architecture	14
3.5.1 CNN Branch	15

3.5.2	MLP Branch	16
3.5.3	Fusion and Classification Head	16
3.6	Training Procedure	17
4	Experiments	18
4.1	Experimental Infrastructure	18
4.2	Evaluation Metrics	18
4.2.1	Classification Metrics	18
4.2.2	Solver-Quality Metrics	19
4.2.3	Computational Cost	19
4.2.4	Non-Learning Baselines	20
4.3	MM-AutoSolver Replication	20
4.4	Single-Channel Experiments	22
4.4.1	Campaign 1: Single-Channel Baseline	22
4.4.2	Campaign 2: Extended Features and Convergence Penalty	23
4.5	Dual-Channel Experiments	23
4.5.1	Multimode HDF5 and Rendering Pipeline	23
4.5.2	Campaign 3: Dual-Channel Approach	23
4.5.3	Campaign 4: Dual-Channel with Extended Features	23
4.6	Ensemble Evaluation	24
4.6.1	Campaign 5: Ensemble Method	24
5	Results and Evaluation	25
5.1	Overview of All Experimental Results	25
5.2	Impact of Image Mode	26
5.3	Impact of Feature Engineering and Convergence Penalty	28
5.4	Single-Channel vs. Dual-Channel	29
5.4.1	Dual-Channel Results Overview	30
5.5	Effect of Image Resolution	32
5.6	Ensemble vs. Single Best Model	33
5.7	Comparison to MM-AutoSolver	34
5.8	Per-Solver Analysis	35
5.9	Practical Inference Cost	36
6	Conclusion	38
6.1	Summary of Findings	38
6.2	Contributions	38
6.3	Limitations	38
6.4	Future Work	38

Contents

List of Figures	39
List of Tables	40
Bibliography	42

Abbreviations

1 Introduction

1.1 Motivation

Linear systems of equations in the form $Ax = b$ arise across nearly every domain in computer science, from structural mechanics, fluid dynamics to machine learning and network analysis. While small systems can be solved directly with Gaussian elimination in $\mathcal{O}(n^3)$ time, real world problems often generate sparse matrices with thousands or millions of unknowns where direct methods become unfeasible. Iterative solvers, such as Conjugate Gradient (CG), Generalized Minimal Residual (GMRES), or Flexible Generalized Minimal Residual (FGMRES), are practical alternatives, they exploit sparsity and can converge in far fewer operations than their counterparts, but their performance depends on the combination of solver and preconditioner chosen for a given problem.

Selecting the best solver/preconditioner pair for a concrete problem is not easy. A configuration that converges in hundreds of iterations for a symmetric positive definite system from a finite-element discretisation may stagnate entirely on an indefinite or unsymmetric problem. Practitioners today rely on mathematical knowledge, trial and error, or specific benchmarking, which require significant manual effort in every step. Automating or improving this selection step is therefore an important open problem, which this thesis addresses.

1.2 Research Gap and Problem Statement

Recent work has shown that machine learning models can predict good solver-preconditioner pairs from matrix properties alone. In particular, Xiong et al. [Xio+25] proposed *MM-AutoSolver*, a multimodal CNN+MLP framework that encodes both a sparsity-pattern image and a hand-crafted feature vector to classify the best solver. They report 78.54% accuracy on their own benchmark dataset of 621 SuiteSparse matrices.

Several gaps remain, however:

- **Limited feature study.** MM-AutoSolver uses 17 fixed features; the sensitivity to feature choice and the benefit of additional structural descriptors (e.g. diagonal dominance variance, structural symmetry) has not been studied.

- **Single image mode.** Only a density representation at 128 px is evaluated; it is unknown whether other encodings (magnitude, sign, RCM-reordered variants) carry complementary information.
- **No multi-channel fusion.** Combining two image modes as a dual-channel input has not been explored.
- **Convergence failures unpenalised.** Standard cross-entropy loss does not discourage predictions that select non-converging solvers; failure rates up to 75% were observed in preliminary experiments.
- **No ensemble evaluation.** The benefit of averaging predictions from multiple independently trained models has not been quantified.
- **Framework integration.** The supervisor’s PETSc-based solver-evaluation framework currently lacks an automated selection component; the extent to which a data-driven model can fill this role is an open question.

1.3 Research Goals and Contributions

This thesis pursues the following goals:

1. **Replication.** Re-implement MM-AutoSolver as a controlled baseline to verify and contextualise the reported results on an independently generated dataset.
2. **Feature engineering.** Extend the feature set from 14 to 20 matrix descriptors and measure the impact on classification accuracy.
3. **Image mode study.** Train and compare 14 single-channel image representations (binary, density, magnitude, sign, symmetry, diagonal, signed_magnitude, and their RCM-reordered counterparts) at 64 px and 128 px.
4. **Dual-channel CNN.** Fuse pairs of the best-performing modes into a two-channel input and evaluate all 15 pairwise combinations at both resolutions.
5. **Convergence penalty.** Introduce a loss term that penalises probability mass assigned to non-converging solvers and evaluate its effect on failure rate.
6. **Ensemble evaluation.** Average softmax outputs from multiple top models and measure the accuracy gain over any individual model.
7. **Framework contribution.** Distil findings into concrete recommendations for integrating automated solver selection into the supervisor’s framework.

1.4 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 surveys existing work on iterative solver selection and the MM-AutoSolver baseline. Chapter 3 describes the dataset, model architecture, training procedure, and evaluation protocol. Chapter 4 details each experimental campaign in turn. Chapter 5 analyses and compares the results across all configurations. Chapter 6 summarises the findings, discusses limitations, and outlines directions for future work.

2 Related Work

2.1 Iterative Solvers and Preconditioners

2.2 Automated Solver Selection

2.3 MM-AutoSolver

2.4 The Supervisor's Solver-Evaluation Framework

2.5 Image Representations of Sparse Matrices

2.6 Convolutional Neural Networks for Classification

3 Methodology

This chapter covers which problem the thesis tackles, how the data used within the different experiments is generated, what the actual data looks like, which solver-preconditioner classes are used, which features are selected and why and also describes the architecture used inside the PyTorch model.

3.1 Problem Formulation

The goal of the thesis is to evaluate the usability of CNNs for a pipeline that tries to predict the solver-preconditioner pair that minimises solve time subject to convergence in a large sparse equation in the form of $Ax = b$, where $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^{n \times 1}$, and $x \in \mathbb{R}^{n \times 1}$. To determine the usability an architecture similar to that in the MM-AutoSolver [Xio+25] paper is introduced initially, which is then modified in certain aspects like the chosen features and the approach of the CNN implementation. These changes are done with the goal in mind to achieve the best possible prediction outcome while also improving the overall failure rates and average speedups of the resulting models which allows them to be used in a complete pipeline.

3.2 Dataset Construction

The dataset is constructed by combining SuiteSparse matrices with synthetically generated matrices and saves the information needed for further processing into an HDF5 file. For the dataset a cap of winning solver-preconditioner pairs is introduced to counter a certain bias towards overrepresented pairs in the training phase. For the dataset used in the experiments this cap was 600, while the minimum of wins for a pair was 250.

3.2.1 Synthetic Data Generation

The synthetic data generation evolved while testing and generating datasets, in the end 37 different generator buckets were defined in the process. These focus on generating matrices with structural properties for which specific solver-preconditioner pairs are theoretically expected to perform well; the actual winner is always determined empirically

by benchmarking [Saa03]. Table 3.1 lists all generator functions grouped by structural property. In practice, however, this worked only partially, as even though some solver wins increased, in most cases the intended target solvers were still outperformed by certain more dominant solver-preconditioner pairs. A more systematic bucket design falls outside the scope of this thesis and these downsides are therefore accepted as a limitation. For generators where only the size changes which solver-preconditioner pair has the highest chance of winning, different buckets that generate specific sizes with a certain generator function are introduced. If these generators produce large matrices, they have a lower chance to be used as they dramatically increase the runtime of the data generation. In the current state, the generators generate matrices with n ranging from 100 to 125,000.

Table 3.1: Synthetic generator buckets grouped by structural matrix property. Size variants of the same generator function are merged into one row. Solver requirements follow [Saa03]: CG requires SPD (Ch. 6), MINRES/SYMMLQ require symmetry (Ch. 6), GMRES and BiCGS variants apply to nonsymmetric systems (Ch. 7). Primary targets are theoretically motivated and empirically confirmed.

Generator function	n range	Primary target(s)
<i>Symmetric positive-definite (SPD)</i>		
random_spd	100–1 000	cg+{ilu, eisenstat, bjacobi}
poisson_2d	100–90 000	cg+*, minres+gamg, fcg+gamg
poisson_3d	125–125 000	cg+*, minres+gamg, fcg+gamg
anisotropic_poisson_2d	100–40 000	cg+ilu, fcg+gamg
anisotropic_poisson_3d	1 700–27 000	fcg+gamg
<i>Symmetric indefinite</i>		
shifted_spd	100–10 000	symmlq+*, cr+*
helmholtz_2d	400–10 000	symmlq+*, minres+gamg
shifted_poisson_2d	400–22 500	symmlq+sor
sym_banded_indefinite	1 000–12 000	symmlq+sor
sym_banded_mild	800–18 000	symmlq+sor
sym_indef_deep	200–12 000	symmlq+jacobi
<i>Nonsymmetric</i>		
random_nonsymmetric	100–20 000	fbcgsl+jacobi, bcgsl+none
convection_diffusion_2d	100–63 000	fbcgsl+ilu, cgs+gamg, fgmres+gamg
convection_diffusion_3d	1 000–43 000	fbcgsl+ilu, bcgsl+asm, cgs+gamg
random_banded	500–20 000	cr+ilu, cg+ilu
barely_dominant_nonsym	200–8 000	dgmres+none

Regarding the benchmarking, all solver pairs are run against a matrix after its generation. A specific timeout is set, after which a solver pair is marked as non-converging, also if the maximum number of iterations is exceeded the pair is marked as non-converging. Some solvers do not work, or only work, on symmetric matrices, therefore they are not run on these specific ones. A pair is valid and gets saved with its spent time if the result is within a certain tolerance. The label a matrix gets in the end

is only the fastest solver-preconditioner pair, the so-called winner.

For each matrix the matrix itself, the features and all the pairs with solving time or an entry for non-convergence are stored.

3.2.2 SuiteSparse Benchmark Matrices

The SuiteSparse dataset generation differs only in the storage of the matrices. Since they are downloaded and stored as .mtx files, the storage of the full matrices in the HDF5 dataset file does not make sense and is therefore omitted to save space. A separate folder for the mtx files is used as a cache which reduces storage requirements while keeping the dataset portable.

3.2.3 Dataset Statistics

The dataset contains 9711 matrices in total, split into 8742 training and 969 validation samples (90/10 stratified split, seed 42). Of these, 559 (5.8%) originate from SuiteSparse; the remainder are synthetically generated.

Table 3.2 shows the per-class sample counts: 13 of the 19 solver-preconditioner pairs reached the cap of 600 samples each, while the remaining six are rarer winners in the benchmarking and therefore underrepresented.

Table 3.2: Class distribution in the full dataset (9711 samples). The label of each matrix is the fastest converging solver-preconditioner pair. Classes are capped at 600.

Rank	Solver + Preconditioner	Count	%
1	cr+ilu	600	6.2
2	cg+eisenstat	600	6.2
3	cg+bjacobi	600	6.2
4	fbcgsl+jacobi	600	6.2
5	gmres+gamg	600	6.2
6	fgmres+gamg	600	6.2
7	cg+ilu	600	6.2
8	cr+jacobi	600	6.2
9	minres+gamg	600	6.2
10	fbcgsl+ilu	600	6.2
11	cr+eisenstat	600	6.2
12	bcgsl+none	600	6.2
13	symmlq+icc	600	6.2
14	bcgsl+asm	428	4.4
15	dgmres+none	354	3.6
16	cgs+gamg	336	3.5
17	fcg+gamg	290	3.0
18	symmlq+jacobi	253	2.6
19	symmlq+sor	250	2.6
Total		9711	100.0

Figure 3.1 shows the distribution of matrix sizes in the main experiment dataset. The sizes span a wide range, from 99 to 125,000 rows, with a median of 8,099 and a mean of 15,046, and a right-skewed distribution driven by the smaller number of very large matrices.

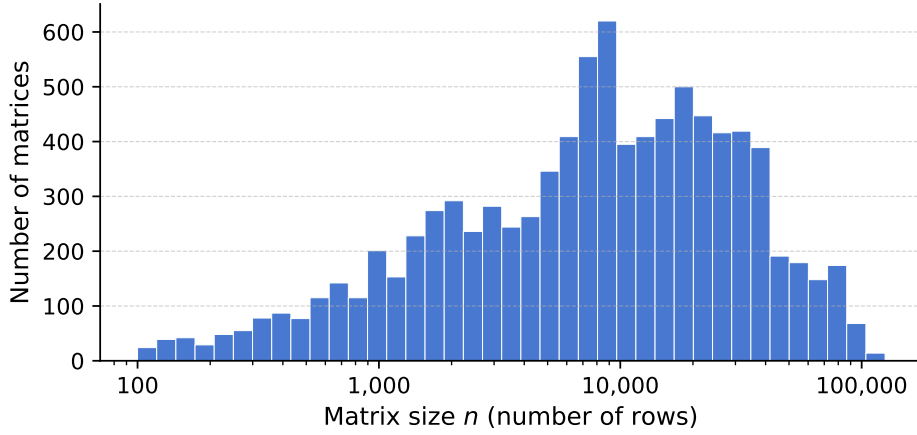


Figure 3.1: Distribution of matrix sizes (number of rows n) for the main thesis dataset (9,711 matrices, Campaigns 1–4). The x -axis uses a logarithmic scale.

3.3 Feature Engineering

The matrix features provided to the MLP branch complement the visual information captured by the CNN, and the choice of features can independently influence prediction quality. The next section introduces the feature sets used in this thesis and how they differ from the MM-AutoSolver approach.

3.3.1 MM-AutoSolver Reference Features

The MM-AutoSolver paper [Xio+25] extracts 17 numerical features from each matrix, listed in Table 3.3. These features are used in the re-implemented baseline (`mm_baseline`) to allow a direct comparison under the paper’s original feature assumptions.

Table 3.3: 17 numerical features used in MM-AutoSolver [Xio+25] (Table 2).

#	Name	Description
0	row_num	Number of rows n
1	nnz	Total number of non-zeros
2	nnz_ratio	Fill ratio nnz/n^2
3	nnz_lower	Non-zeros in strict lower triangle
4	nnz_upper	Non-zeros in strict upper triangle
5	nnz_diagonal	Number of non-zero diagonal entries
6	ave_nnz_row	Average non-zeros per row (nnz/n)
7	max_nnz_row	Maximum non-zeros in any single row
8	arr_nnz_rows	Variance of per-row non-zero counts
9	max_value	Maximum absolute off-diagonal entry
10	max_value_diagonal	Maximum absolute diagonal entry
11	diagonal_dominance_ratio	Fraction of rows where $ a_{ii} > \sum_{j \neq i} a_{ij} $
12	is_symmetry	1 if $\ A - A^\top\ _F / \ A\ _F < 0.01$, else 0
13	pattern_symmetry	Fraction of non-zero positions mirrored in $A^\top$
14	value_symmetry	$1 - \ A - A^\top\ _F / \ A\ _F$
15	row_variability	$\log(1 + \max_i(\max_j a_{ij} / \min_j a_{ij}))$
16	col_variability	Same as above computed column-wise

3.3.2 Original 14 Features

The 14 features extracted from each matrix, shown in Table 3.4, are grouped by the matrix property they capture. Features 0–2 describe size and sparsity (matrix dimension, non-zero count, and fill ratio). Feature 3 captures symmetry via the asymmetry ratio. Features 4 and 9 characterise diagonal dominance, which is relevant for the stability of ILU-based preconditioners [MV77; Saa03]. Features 5–8 encode magnitude and scaling information, and features 10–13 capture structural properties such as bandwidth, diagonal fill, and row-norm variation. While the covered property groups overlap with those in MM-AutoSolver, the specific formulations differ in order to explore whether alternative encodings of the same matrix characteristics lead to better solver predictions.

3.3.3 Extended 20-Feature Set

During experimentation, six additional features were introduced to address specific prediction weaknesses observed in Campaign 1, expanding the feature vector to 20

Table 3.4: Original 14 matrix features used in the single-channel experiments.

#	Name	Description
0	$\log(1 + n)$	Matrix size (number of rows)
1	$\log(1 + \text{nnz})$	Number of non-zero entries
2	nnz/n^2	Density (fill ratio)
3	$\ A - A^\top\ _F / \ A\ _F$	Asymmetry measure
4	$\text{mean}(d_i / \sum r_i)$	Mean diagonal dominance
5	$\ A\ _F / n$	Frobenius norm per row
6	$\text{tr}(A) / n$	Normalised trace
7	$\max a_{ij} / \text{mean} a_{ij} $	Relative maximum entry
8	$\log(1 + \rho(A))$	Log spectral radius
9	$\log(1 + \kappa_{\text{diag}})$	Diagonal condition number proxy
10	bw / n	Normalised bandwidth
11	$\text{nnz}_{\text{diag}} / n$	Fraction of non-zero diagonal entries
12	$\text{CV}(\ r_i\)$	Row norm coefficient of variation
13	$\ A - D\ _F / \ A\ _F$	Off-diagonal Frobenius fraction

entries (Table 3.5). Features 14 and 15 improve the separation between CG-type and GMRES-type solvers by encoding the sign structure of off-diagonal entries and the fraction of positive diagonal entries, both of which correlate with matrix definiteness [Saa03]. Feature 16 distinguishes structurally symmetric from truly asymmetric matrices, helping to separate gmres from fgmres predictions. Features 17 and 19 target ILU preconditioner stability: the minimum per-row diagonal dominance detects the single worst-case row for ILU breakdown, while the variance of dominance ratios captures patchy instability that the mean alone misses. Feature 18 encodes whether the non-zero structure is grid-like or unstructured, which is a strong predictor of GAMG suitability [Stü01].

Table 3.5: The 6 additional matrix features introduced in Campaign 2, expanding the feature vector from 14 to 20 entries.

#	Name	Description
14	neg_offdiag_frac	Fraction of off-diagonal entries that are negative
15	pos_diag_frac	Fraction of strictly positive diagonal entries
16	struct_sym_frac	Fraction of (i, j) entries with a matching (j, i) entry
17	min_diag_dominance	$\min_i d_i / \sum a_{ij} _{j \neq i}$
18	nnz_per_row_cv	Coefficient of variation of per-row non-zero counts
19	diag_dom_variance	Variance of per-row diagonal dominance ratios

3.4 Image Representations

Image representation is an interesting topic within the thesis approach of improving prediction accuracy, as different downsampling methods store completely different data within the image later read by the CNN. Thus, several different downsampling methods were implemented, as shown in Table 3.6, including the RCM (Reverse Cuthill–McKee) reordering [CM69] method which permutes matrix rows and columns to reduce bandwidth before rendering. This reordering is applied solely for image generation and is not passed to the solver, as the original matrix is used unchanged during benchmarking.

Throughout the thesis, experiments are referred to using the naming convention `<mode>_<size>` for single-channel and `<mode1>_<mode2>_<size>` for dual-channel experiments, where `<size>` is the image resolution in pixels. For example, `magnitude_64` denotes magnitude downsampling at 64 px, and `magnitude__rcm_log_density_128` a dual-channel run with magnitude and RCM log-density modes at 128 px.

3.5 Model Architecture

The model consists of two sub-models combined into one prediction model. The two branches are an MLP that processes previously defined matrix features, and a CNN that processes pre-rendered images. Outputs are then concatenated and passed to a shared classification head which produces the probability distribution over the 19 solver-preconditioner pairs. Concatenation was chosen over element-wise addition as it is the more common fusion practice [BAM18] and gives the classification head greater freedom in weighting contributions from each branch independently. Also two different model sizes are introduced to evaluate the effect of capacity. The CNN branch

Table 3.6: The 14 image modes evaluated in the single-channel experiments. The six `rcm_*` variants apply Reverse Cuthill–McKee reordering to the matrix before rendering with the corresponding base mode.

Mode	Description
<code>binary</code>	1 where any non-zero falls in a pixel cell, 0 elsewhere
<code>density</code>	Non-zero count per cell divided by cell area, normalised to $[0, 1]$
<code>log_density</code>	$\log(1 + \text{count})$ per cell, normalised to $[0, 1]$
<code>magnitude</code>	Mean absolute entry value per cell, normalised to $[0, 1]$
<code>sign</code>	Mean sign per cell: positive $\rightarrow 1$, negative $\rightarrow 0$, mixed $\rightarrow 0.5$
<code>signed_magnitude</code>	$\text{sign}(a) \cdot \log(1 + a)$ per cell, normalised to $[0, 1]$ with $0.5 = 0$
<code>symmetry</code>	$ A - A^\top $ per cell, normalised to $[0, 1]$
<code>diagonal</code>	Off-diagonal entries = 0.5, diagonal entries = 1.0
<code>rcm_binary</code>	RCM-reordered variants of the above modes. Reverse Cuthill–McKee reordering reduces matrix bandwidth before rendering, grouping structurally related entries closer to the diagonal.
<code>rcm_density</code>	
<code>rcm_log_density</code>	
<code>rcm_magnitude</code>	
<code>rcm_sign</code>	
<code>rcm_signed_magnitude</code>	

supports a single image channel and a dual one where 2 different images can be used as input.

3.5.1 CNN Branch

The CNN branch accepts an image of the size $(C \times H \times W)$ as input, where $C = 1$ for single-channel and $C = 2$ for dual-channel experiments, and $H = W \in \{64, 128\}$ depending on the image resolution.

The branch begins with an initial convolution block, a 3×3 convolution with c output channels, followed by batch normalisation and ReLU activation, and a 2×2 max-pooling layer that halves the spatial dimensions. This is then followed by two residual blocks (three for the large model variant), where each one is separated by a 2×2 max-pooling step. Each residual block consists of two 3×3 convolutions with batch normalisation and a skip connection: $\mathbf{y} = \text{ReLU}(\mathbf{x} + \text{block}(\mathbf{x}))$.

After the convolutional stack, an adaptive average pooling layer reduces the spatial dimensions to a fixed 4×4 output regardless of input resolution. The result is then flattened and passed through a fully connected layer to produce a fixed-size CNN embedding of dimension d_{CNN} .

Two model sizes are defined:

- **Small:** $c = 64$ channels, 2 residual blocks, $d_{\text{CNN}} = 256$, dropout $p = 0.3$, $\approx 0.5\text{M}$ parameters.
- **Large:** $c = 128$ channels, 3 residual blocks, $d_{\text{CNN}} = 512$, dropout $p = 0.4$, $\approx 2\text{M}$ parameters.

3.5.2 MLP Branch

The MLP branch processes the vector of scalar matrix features $\mathbf{f} \in \mathbb{R}^{n_{\text{feat}}}$, where $n_{\text{feat}} \in \{14, 20\}$ depending on the experimental campaign.

It consists of two fully connected layers with ReLU activations:

$$\mathbf{h} = \text{ReLU}(W_2 \text{ReLU}(W_1 \mathbf{f} + b_1) + b_2), \quad W_1, W_2 \in \mathbb{R}^{d_{\text{MLP}} \times \cdot}$$

The hidden dimension is $d_{\text{MLP}} = 64$ (small model) or $d_{\text{MLP}} = 128$ (large model). No dropout is applied in this branch. The features are already low-dimensional and the risk of overfitting here is low.

3.5.3 Fusion and Classification Head

The outputs of the two branches are concatenated:

$$\mathbf{z} = [\mathbf{c}_{\text{CNN}}; \mathbf{h}_{\text{MLP}}] \in \mathbb{R}^{d_{\text{CNN}} + d_{\text{MLP}}}$$

and passed through a two-layer classification head with ReLU activation and dropout, producing logits over the 19 solver-preconditioner classes:

$$\hat{\mathbf{y}} = W_{\text{out}} \text{ReLU}(W_{\text{head}} \mathbf{z} + b_{\text{head}}) + b_{\text{out}}$$

The head hidden dimension is 128 (small) or 256 (large).

This concatenation-based fusion differs from the MM-AutoSolver [Xio+25] approach, which uses an element-wise addition of the CNN and MLP outputs. Concatenation is more flexible as the classification head here can learn arbitrary weighting of the two branches, whereas element-wise addition requires both branches to produce outputs with the same size and a compatible representation.

When `no_cnn=True`, the CNN branch is disabled entirely and only the MLP branch feeds into the head, serving as the `nocnn` feature-only baseline.

3.6 Training Procedure

All models are trained with the AdamW optimiser [LH19] at a learning rate of 3×10^{-4} and weight decay of 10^{-3} . A cosine annealing schedule reduces the learning rate to zero over the full training run of 256 epochs. Batches of size 256, or in the dual-channel case 64 due to memory restrictions, are drawn from the training split with shuffling.

The base loss is cross-entropy over the 19 classes. In Campaigns 2 and 4 an additional convergence penalty is added:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \sum_{k \notin \mathcal{C}(A)} \hat{p}_k$$

where $\mathcal{C}(A)$ is the set of solvers that converged for matrix A , $\hat{p}_k$ is the predicted probability for solver k , and $\lambda = 1.0$. This directly penalises probability mass assigned to diverging solvers and is expected to reduce failure rates.

The dataset is split into 90% training and 10% validation data using a stratified split with seed 42, ensuring each class is represented proportionally in both splits. Checkpoints are saved every 5 epochs and the final epoch checkpoint is used for evaluation.

4 Experiments

This chapter presents the experiments conducted within the thesis and describes the key design choices and the motivation behind each campaign. Detailed results and metric comparisons are evaluated in Chapter 5.

4.1 Experimental Infrastructure

To ensure reproducibility the whole pipeline runs within a Docker environment. The pipeline includes distinct scripts for data generation, model training and model evaluation. The training data is stored in HDF5 file format to enable fast access, and is shared across experiments unless stated otherwise. Its generation was performed on an AMD Ryzen AI 7 350 / Radeon 860 and an AMD Ryzen 5 1600 Six-Core, described in more detail in Section 3.2 Checkpoints and logs are namespaced by experiment name and image size, allowing multiple runs to coexist without overwriting. The training of the model for the different experiments was done on an NVIDIA GeForce RTX 5060 Laptop GPU. All runs use a fixed random seed for reproducibility.

The full source code, configuration files, experiment logs and LaTeX sources that were used for the thesis are publicly available on GitHub under the "Thesis-Submission" Release ¹

4.2 Evaluation Metrics

4.2.1 Classification Metrics

The experiment results are measured with four classification metrics, following the evaluation protocol of MM-AutoSolver [Xio+25], to allow direct comparison:

- **Accuracy (Acc)** - the fraction of test samples for which the predicted solver-preconditioner pair matches the best solver.
- **Macro Precision (MP)** - the average per-class precision across all 19 classes, weighted equally regardless of class size.

¹...

- **Macro Recall (MR)** - the average per-class recall, weighted equally across all classes.
- **Macro F1 (F1)** - the harmonic mean of macro precision and macro recall, balancing both measures into a single score.

Macro averaging is used rather than weighted averaging to treat all solver classes equally, which is important given the imbalanced class distribution in the dataset.

4.2.2 Solver-Quality Metrics

If a predicted solver-preconditioner pair is not the best result but comes in as a close second it won't be recognised for the accuracy which may not be ideal when targeting the implementation of the models within a specific pipeline. For this case two separate metrics are introduced, which also give positive feedback if the chosen solver-preconditioner pair is only marginally slower, just the second-best choice or might be extremely failure-resistant.

- **Top-k accuracy (k=2,3)** - probability the predicted solver is within the top-k predictions. Top-3 accuracy is used as the primary quality metric in the results discussion, with near-optimal values available in the experiment summary files.
- **Near-optimal accuracy (k=5,10,20)** - the predicted solver is within k% of the best solver time.
- **Mean Runtime Ratio (MRT $\times$)** - the average ratio of the predicted solver's runtime to the best solver's runtime. A value of 1.0 means the predicted solver is always optimal; higher values indicate how much slower the predictions are on average. Samples where the predicted solver did not converge are excluded from the MRT calculation and counted in the failure rate instead, so the two metrics should be read together.
- **Failure rate** - the fraction of test samples where the predicted solver did not converge. This matters in practice because a non-converging solver must run to timeout before the failure is detected, which can be more costly than simply picking a slower but converging solver.

4.2.3 Computational Cost

Other metrics that need to be introduced regard the time cost within data generation, model training and per-matrix feature and image generation. Therefore, training

time and inference costs are also logged for each experiment and can be seen in corresponding summary files. For the data generation no specific time cost was logged, unfortunately.

4.2.4 Non-Learning Baselines

Two non-learning baselines are introduced as reference points. **1-NN** and **5-NN** are k -nearest-neighbour classifiers that predict the best solver by finding the most similar training matrices in the scalar feature space (14-dimensional in Campaigns 1 and 3, 20-dimensional in Campaigns 2 and 4), without any CNN branch. Additionally, a **nocnn** baseline disregards the CNN branch entirely, learning only from the feature vector through the MLP and classification head. This is used to isolate the contribution of the image input and allows the CNN’s value to be assessed independently of the feature branch.

4.3 MM-AutoSolver Replication

The first experiment replicates the results of the MM-AutoSolver [Xio+25] paper with a dataset that resembles the dataset from the paper in solver-preconditioner distribution. Due to the computational cost of benchmarking all 19 solver pairs, which scales significantly with matrix size, the replication dataset is limited to matrices with up to 84,617 rows, compared to over one million in the original paper. Table 4.1 compares the per-class sample counts of the replication dataset against the actual numbers of the paper.

Since the generator buckets were less targeted in the early stages, generation of certain classes took a significant amount of time. Therefore, different classes are over- and underrepresented and the dataset remains skewed, but in different directions than the paper.

Table 4.1: Per-class sample counts: MM-AutoSolver paper dataset vs. replication dataset used in this thesis.

Solver	Ours	Ours %	Paper	Paper %
fbcgsl+jacobi	2519	10.9	2173	18.7
bcgsl+none	2502	10.8	2054	17.7
symmlq+icc	162	0.7	1201	10.3
symmlq+jacobi	211	0.9	923	7.9
dgmres+none	43	0.2	650	5.6
gmres+gamg	394	1.7	640	5.5
cr+eisenstat	2505	10.8	598	5.1
symmlq+sor	109	0.5	582	5.0
fbcgsl+ilu	1247	5.4	562	4.8
minres+gamg	412	1.8	524	4.5
fcg+gamg	143	0.6	342	2.9
cr+jacobi	2503	10.8	310	2.7
cg+ilu	2511	10.8	275	2.4
fgmres+gamg	146	0.6	226	1.9
cg+eisenstat	2525	10.9	224	1.9
cg+bjacobi	2503	10.8	193	1.7
cr+ilu	2500	10.8	68	0.6
cgs+gamg	106	0.5	49	0.4
bcgsl+asm	126	0.5	29	0.2
Total	23,167	100.0	11,623	100.0

Figure 4.1 shows the distribution of matrix sizes in the replication dataset, with sizes ranging from 100 to 84,617 rows and a median of 1,485.

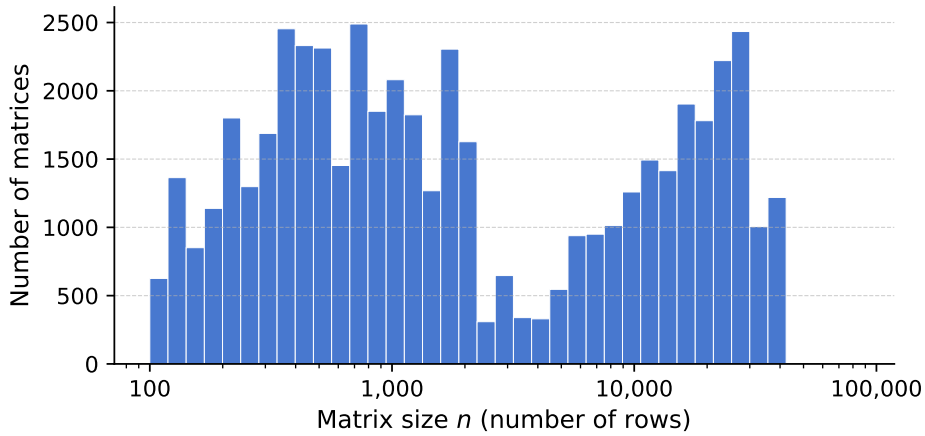


Figure 4.1: Distribution of matrix sizes (number of rows n) for the MM-AutoSolver replication dataset (23,167 matrices). The x -axis uses a logarithmic scale.

With this dataset the experiment was set up with a 128×128 pixel density image for the CNN branch, following the paper, their proposed 17 features are also calculated and used. The re-implementation uses the original MM-AutoSolver architecture, two convolutional layers with Tanh activation and element-wise CNN/MLP fusion, rather than the thesis model (Section 3.5). For the training, a learning rate of 10^{-3} , 256 training epochs and a batch size of 512 are set.

4.4 Single-Channel Experiments

This section describes the different experiments done with a single CNN channel active. They differ by their feature vector, the activation of the non-convergence penalty and model size.

4.4.1 Campaign 1: Single-Channel Baseline

The first experiment on the single branch is used to create a baseline for future experiment runs. The 14 different image modes (Section 3.4) are tested to determine which of them carries the most useful information, and they are run on both 64px and 128px images. Convergence penalty is deactivated and the small model (Section 3.5) is used to establish a clean baseline.

4.4.2 Campaign 2: Extended Features and Convergence Penalty

Campaign 2 addresses two weaknesses observed in Campaign 1. Some solvers had especially low accuracy and F1 values, therefore an additional 6 features described in Table 3.5 were introduced. To reduce high failure rates observed in certain classes the convergence penalty (Section 3.6) with $\lambda = 1.0$ is used. The whole experiment is run on the large model (Section 3.5) and uses the same image modes as Campaign 1 but only an image size of 64 px.

4.5 Dual-Channel Experiments

This section describes the experiments that use the dual-channel CNN approach. To reduce computation time, only the best single-channel approach modes are combined with additional combinations included where mode pairs are expected to capture complementary matrix information.

4.5.1 Multimode HDF5 and Rendering Pipeline

To evaluate dual-channel experiments efficiently, all required image modes are pre-rendered into a single shared HDF5 file rather than maintaining separate per-experiment datasets. The file stores one dataset per image mode under the key `images_<mode>`, alongside the shared features, labels and runtimes from the original dataset.

4.5.2 Campaign 3: Dual-Channel Approach

The experiment setup is similar to the one in Campaign 1, except this time a different set of image modes and the large model (Section 3.5) are used and the image sizes are fixed at 64 px. The top 6 downsampling methods by F1 value are selected from Campaign 1 (Subsection 4.4.1), as well as the sign mode. These 7 methods are then combined into all their pairwise combinations and tested. This results in 21 different image mode combinations that can be seen in Table 5.3.

4.5.3 Campaign 4: Dual-Channel with Extended Features

This campaign combines the dual-channel approach with the improvements introduced in Campaign 2 (Subsection 4.4.2) with convergence penalty $\lambda = 1.0$, the 20-feature set and the large model (Section 3.5). The image modes are selected based on F1 performance in Campaign 2, and all selected mode pairs are run on both 64 px and 128 px. This results in 15 different image mode combinations each run at two image sizes, therefore 30 different models.

4.6 Ensemble Evaluation

4.6.1 Campaign 5: Ensemble Method

The ensemble combines independently trained models by averaging their softmax probability vectors before taking the argmax prediction. No retraining is required as the member models are loaded from their saved checkpoints and are evaluated on the shared validation split.

The shared multimode HDF5 (Subsection 4.5.1) ensures that all members receive identical image data during ensemble inference. Member selection follows two criteria: high individual F1 and complementary second channels, so that the ensemble covers a wider range of matrix characteristics than any single model.

For Campaign 3 the four ensemble members are (all 64 px):

- magnitude+signed_magnitude
- magnitude+rcm_signed_magnitude
- magnitude+symmetry
- rcm_magnitude+signed_magnitude

For Campaign 4 the evaluated ensemble uses four members (all 128 px):

- magnitude+log_density
- magnitude+rcm_log_density
- rcm_magnitude+rcm_signed_magnitude
- symmetry+log_density

These were selected for complementary first channels (magnitude $\times 2$, rcm_magnitude, symmetry) and non-redundant second channels.

5 Results and Evaluation

This chapter presents and interprets the quantitative outcomes of the experiments described in Chapter 4. Each section addresses one research dimension, image mode choice, feature engineering, single- vs. dual-channel, image resolution, and ensembling, before placing the best results in context with the MM-AutoSolver baseline.

5.1 Overview of All Experimental Results

Each of the following sections presents its results table alongside the discussion, introducing metrics as they become relevant. Bold marks the best value in each column; for Fail% and $\text{MRT}\times$, lower is better.

5.2 Impact of Image Mode

Table 5.1: Campaign 1 — Single-channel: 14 features, no convergence penalty, small model.

Experiment	Acc%	MP%	MR%	F1%	Top-2%	Top-3%	Fail%	MRT×
1-NN	60.06	58.94	58.56	57.97	82.04	90.30	2.27	1.533
5-NN	62.95	62.46	61.85	61.12	82.56	90.40	3.20	1.605
nocnn	58.31	60.66	61.14	57.57	78.95	86.38	5.88	1.345
binary_64	59.55	63.33	62.69	58.80	82.04	90.61	6.30	1.274
binary_128	60.06	63.65	62.95	59.26	83.28	91.95	5.47	2.819
density_64	60.27	63.17	62.29	59.46	81.42	91.54	5.47	1.291
density_128	63.26	65.25	65.57	62.25	84.42	93.19	4.44	1.271
diagonal_64	59.75	62.92	62.82	58.79	82.35	90.92	5.37	4.196
diagonal_128	59.44	63.07	62.53	58.79	81.63	90.51	5.99	1.316
log_density_64	62.33	65.39	64.86	61.38	82.97	92.88	5.16	2.807
log_density_128	62.64	64.33	64.98	61.69	83.90	92.67	5.16	1.271
magnitude_64	64.19	66.19	66.55	63.42	86.58	93.91	3.92	1.259
magnitude_128	63.88	67.58	66.35	62.99	86.48	93.81	3.92	1.115
sign_64	60.37	63.08	62.59	59.29	82.25	91.64	5.68	1.288
sign_128	61.20	65.11	64.28	60.53	83.08	92.16	4.02	4.139
signed_magnitude_64	62.85	64.56	65.20	61.68	84.83	94.22	4.75	1.069
signed_magnitude_128	61.82	64.75	64.53	60.42	84.73	93.09	5.06	1.222
symmetry_64	62.95	64.40	65.66	61.85	85.66	94.01	3.51	4.081
symmetry_128	62.95	64.14	65.32	61.62	85.96	94.43	4.64	1.233
rcm_binary_64	60.06	62.61	62.84	59.17	82.46	91.64	6.40	1.322
rcm_binary_128	59.86	62.18	62.40	59.01	82.35	91.64	6.81	1.269
rcm_density_64	61.40	64.53	64.09	60.43	83.59	93.40	5.68	1.235
rcm_density_128	62.33	65.47	64.84	61.33	82.77	93.29	4.75	1.262
rcm_log_density_64	62.85	65.78	65.32	61.88	83.90	93.29	4.75	2.770
rcm_log_density_128	61.82	63.55	64.19	60.94	83.49	92.36	5.47	1.271
rcm_magnitude_64	62.85	65.97	65.50	62.14	84.83	93.19	4.64	1.288
rcm_magnitude_128	63.05	65.93	65.35	62.55	84.62	92.67	4.13	1.281
rcm_sign_64	61.71	65.19	64.23	60.84	82.56	92.47	4.64	2.802
rcm_sign_128	61.82	65.53	64.63	61.05	82.77	92.98	4.23	2.829
rcm_signed_magnitude_64	63.67	66.12	65.51	62.24	85.66	94.01	4.02	2.586
rcm_signed_magnitude_128	63.36	64.74	65.65	62.23	85.35	94.12	4.64	1.235

Across the first two Campaigns where single image modes were used, the magnitude_64 downsampling is the strongest single predictor. It consistently outperforms its 13 competitors, winning the first campaign with a 64.19% accuracy and a 63.42% F1 value and the second one with a 64.29% accuracy and a 63.38% F1 value.

While in both Campaigns the `magnitude_64` downsampling approach yields better results than their best baseline approaches there are always some configurations where nearly no improvement or even worse values can be observed. In Campaign 1 the `binary_64` model for example has a lower accuracy and only a F1 value 0.03pp higher than the 1-NN baseline, showing that the binary downsampling doesn't store that much valuable information when cropped to small images. In Campaign 2 the `binary_64` model also does not show better results, but within this campaign the `density_64` approach was more notable as it was the only one that dropped by over 8%. While investigating it showed that it stopped predicting the solver-preconditioner pairs `minres+gamg` and `fcg+gamg` which resulted in a failure rate of 0% meaning that the non-convergence penalty works against the model in this case, thus not working as intended in this case.

As only a certain amount of downsampling methods made sense, to introduce additional variations, RCM reordering was applied to selected modes. This had no significant impact in single-channel model results as values in most cases stayed within $\pm 0.6\%$ of the original values.

The worst-performing modes consist of those that only regard structural features like binary and diagonal and do not incorporate any magnitude information, confirming that entry values carry essential information for solver selection. The six highest-ranking modes by F1 - `magnitude`, `rcm_signed_magnitude`, `rcm_magnitude`, `signed_magnitude`, `symmetry`, and `density (128px)` - are therefore carried forward as the basis for the dual-channel experiments in Campaign 3 (Subsection 4.5.2).

The results also show that if a model with exceptional speed needs to be selected, in most cases it won't be the model with the highest accuracy or the best F1 value. In Campaign 1 for example, `magnitude_64` achieves the best accuracy and F1, but `signed_magnitude_64` achieves a lower $\text{MRT} \times$ of 1.069 versus 1.259, making it a better choice when runtime ratio is the priority.

5.3 Impact of Feature Engineering and Convergence Penalty

Table 5.2: Campaign 2 — Single-channel: 20 features, convergence penalty $\lambda = 1.0$, large model, 64 px.

Experiment	Acc%	MP%	MR%	F1%	Top-2%	Top-3%	Fail%	MRT×
1-NN	59.44	59.02	58.34	57.00	82.15	90.40	1.86	1.539
5-NN	63.57	63.25	62.38	61.61	82.97	91.02	2.89	1.589
nocnn	59.75	62.40	62.72	59.08	80.80	89.78	4.95	2.926
binary_64	59.34	62.66	62.53	58.87	81.53	91.33	5.06	1.343
density_64	52.63	55.76	54.56	50.74	72.14	79.26	5.16	1.363
diagonal_64	59.13	62.96	62.11	58.46	81.63	90.61	5.16	1.371
log_density_64	62.23	66.12	65.13	61.65	82.15	92.36	4.54	1.284
magnitude_64	64.29	66.58	66.36	63.38	85.76	93.60	4.23	1.218
sign_64	60.99	64.54	63.49	60.19	83.08	93.60	4.44	1.304
signed_magnitude_64	61.20	63.39	62.78	59.85	85.24	94.01	5.16	1.068
symmetry_64	63.36	64.74	65.49	61.94	85.04	93.81	3.72	1.254
rcm_binary_64	59.03	62.82	62.04	58.30	81.01	90.71	5.99	1.304
rcm_density_64	61.51	65.59	64.19	61.00	84.31	93.91	4.23	1.289
rcm_log_density_64	62.75	66.52	65.27	62.12	82.56	92.26	4.75	1.264
rcm_magnitude_64	63.47	66.80	65.74	62.79	85.35	93.81	3.41	2.766
rcm_sign_64	61.51	65.43	63.82	60.87	81.94	91.95	4.23	1.326
rcm_signed_magnitude_64	62.54	65.17	64.09	61.36	83.90	92.67	4.44	1.069

From Campaign 1 to Campaign 2, six additional matrix features and the convergence penalty (Section 3.6) with $\lambda = 1.0$ were introduced. Because no isolated ablation was run for the 20-feature approach with $\lambda = 0$ and the 14-feature approach with $\lambda = 1.0$, the effects of these changes need to be discussed simultaneously. Note that from this point on, all experiments use the larger model (Section 3.5).

Accuracy impact The accuracy gains from Campaign 1 to Campaign 2 are modest. The nocnn MLP-only baseline approach improves by 1.44% to 59.75%, suggesting that the new features add useful information. Since the penalty primarily shapes which solver classes the model avoids rather than the feature representation itself, the expanded feature set is the more likely driver of this particular gain. For CNN-based models the improvement is negligible, the best approach being magnitude, only shifts 0.10 pp to 64.29% in accuracy. This shows that information from the new features might already be present in some way within the CNN representations, which would explain the diminishing results in the CNN model approaches vs. the nocnn approach.

Failure rate impact The metric where the convergence penalty is most clearly visible is the failure rate. Combined failure rates across single-channel experiments dropped from 3.5%–6.8% in Campaign 1 to 3.4%–6.0% in Campaign 2. This shows that the overall failure rate drops, sometimes significantly. When looking at the per-class results for the magnitude downsampling with 14 features and without the penalty versus the 20 features and penalty approach, before the penalty, 4 classes had failure rates above 10%, after, only 2 remain. Unfortunately, for `symmlq+sol` the failure rate surged from 14.8% to 28.2%, which then drops the overall failure rate back to previous levels.

This behaviour is reproduced in the other models as well, therefore the penalty approach needs to be examined closely as to not only provide a shift in failure rates but realistic improvement.

Density collapse side-effect The main negative side effect in this experiment is the density collapse, where accuracy drops to 52.63% from previously 60.27%. This loss of over 10 pp is only observed in this model and might indicate a training instability, as the drop was not reproduced in later experiments, but may also hint that the density encoding and the penalty gradient do not interact well.

5.4 Single-Channel vs. Dual-Channel

Adding a second CNN channel consistently improves over the single-channel approaches. The best single-channel model, `magnitude_64` (64.29% accuracy, 63.38% F1), is outperformed by the best dual-channel 64 px model `magnitude__log_density_64`, which achieves 65.94% accuracy and 65.02% F1. The best overall model is `magnitude__rcm_log_density_128` with 66.56% accuracy, a 2.27 pp improvement over the best single-channel approach.

The magnitude of the gain depends strongly on which two channels are chosen; when their information encodes different properties they excel the most. This can be seen by the combination of `magnitude` (absolute values) with `log_density` (structural density) or `rcm_log_density` (density after bandwidth reduction). Pairs with similar information like `rcm_log_density__log_density_128` (two log-family channels) often drop even below some single-channel approaches if their single-channel performance was already weak.

This confirms that a second channel is useful but only if the two downsampling methods contain non-redundant information.

Across all dual-channel experiments the most important channel is `magnitude`, as it consistently appears in the best dual-channel pairs, confirming that `magnitude` encodes discriminative information essential for solver selection.

5.4.1 Dual-Channel Results Overview

Table 5.3: Campaign 3 — Dual-channel: 14 features, no convergence penalty, large model, 64px.

Experiment	Acc%	MP%	MR%	F1%	Top-2%	Top-3%	Fail%	MRT×
1-NN	60.06	58.94	58.56	57.97	82.04	90.30	2.27	1.533
5-NN	62.95	62.46	61.85	61.12	82.56	90.40	3.20	1.605
nocnn	60.06	63.20	62.71	59.37	80.91	89.37	6.19	1.289
density__rcm_magnitude_64	64.40	67.28	65.93	63.72	85.66	93.40	5.06	1.219
density__sign_64	63.98	66.90	66.42	63.14	85.35	94.12	4.44	1.217
density__signed_magnitude_64	61.82	64.35	63.66	60.52	84.42	93.19	4.54	1.227
density__symmetry_64	63.57	63.78	66.09	62.98	85.96	94.12	3.61	1.796
magnitude__density_64	64.50	68.09	67.13	63.95	87.41	94.43	4.23	1.215
magnitude__rcm_magnitude_64	64.40	66.64	65.89	63.24	85.24	92.98	4.13	1.244
magnitude__rcm_signed_magnitude_64	66.05	69.25	67.64	64.79	86.79	94.22	2.89	3.920
magnitude__sign_64	64.50	66.79	66.93	63.52	85.96	93.91	4.23	1.215
magnitude__signed_magnitude_64	66.05	68.21	68.23	65.32	86.48	94.12	2.79	4.476
magnitude__symmetry_64	65.63	67.20	67.59	64.66	84.83	93.40	3.61	1.258
rcm_magnitude__sign_64	64.91	66.97	67.04	64.03	85.66	94.22	4.13	1.229
rcm_magnitude__signed_magnitude_64	65.63	68.90	67.61	64.86	85.45	94.22	3.82	1.235
rcm_magnitude__symmetry_64	64.29	66.85	66.74	63.34	85.55	94.53	3.61	3.298
rcm_signed_magnitude__density_64	63.36	65.94	65.10	61.89	84.83	92.67	4.54	1.063
rcm_signed_magnitude__rcm_magnitude_64	63.88	66.95	65.97	63.22	85.66	93.81	4.02	1.221
rcm_signed_magnitude__sign_64	62.44	65.12	64.14	60.79	84.93	93.70	4.33	1.226
rcm_signed_magnitude__signed_magnitude_64	62.33	64.34	63.62	60.55	84.11	93.50	4.64	1.065
rcm_signed_magnitude__symmetry_64	62.85	63.05	64.26	61.81	83.49	92.26	3.82	1.631
signed_magnitude__sign_64	62.33	64.70	63.94	61.29	84.00	93.60	4.64	1.235
symmetry__sign_64	63.05	65.83	65.22	61.92	85.66	93.91	3.72	3.093
symmetry__signed_magnitude_64	63.36	64.72	64.92	62.08	85.96	94.53	3.20	1.930

Table 5.4: Campaign 4 — Dual-channel: 20 features, convergence penalty $\lambda = 1.0$, large model, 64 px and 128 px.

Experiment	Acc%	MP%	MR%	F1%	Top-2%	Top-3%	Fail%	MRT×
1-NN	59.44	59.02	58.34	57.00	82.15	90.40	1.86	1.539
5-NN	63.57	63.25	62.38	61.61	82.97	91.02	2.89	1.589
nocnn	59.75	62.40	62.72	59.08	80.80	89.78	4.95	2.926
magnitude_log_density_64	65.94	68.42	67.73	65.02	86.79	94.63	4.02	1.214
magnitude_log_density_128	65.94	68.67	68.22	65.60	87.00	94.63	4.13	1.051
magnitude_rcm_log_density_64	65.43	67.72	67.75	64.74	86.27	94.63	4.02	1.220
magnitude_rcm_log_density_128	66.56	69.15	68.88	65.53	86.38	94.53	3.51	1.051
magnitude_rcm_magnitude_64	63.78	66.49	65.76	63.37	86.27	94.63	3.92	1.227
magnitude_rcm_magnitude_128	65.02	66.73	66.62	64.22	87.10	94.74	3.61	1.222
magnitude_rcm_signed_magnitude_64	65.53	67.52	67.36	64.30	85.96	93.40	3.72	1.234
magnitude_rcm_signed_magnitude_128	65.84	69.39	67.59	65.16	86.38	94.43	3.72	1.213
magnitude_symmetry_64	64.71	66.34	66.35	63.49	85.45	94.01	2.68	3.821
magnitude_symmetry_128	64.60	66.86	66.41	63.87	85.55	93.81	3.61	1.086
rcm_log_density_log_density_64	62.02	66.30	65.07	61.44	82.66	92.47	4.23	1.276
rcm_log_density_log_density_128	63.26	66.32	65.71	62.28	84.73	94.32	4.64	1.223
rcm_magnitude_log_density_64	64.09	67.24	66.56	63.77	85.66	93.29	3.92	1.247
rcm_magnitude_log_density_128	64.60	67.97	67.00	64.35	86.17	93.91	4.02	1.227
rcm_magnitude_rcm_log_density_64	63.98	66.95	65.98	63.51	85.35	93.40	3.82	1.256
rcm_magnitude_rcm_log_density_128	64.50	68.06	67.03	64.19	84.52	93.19	3.51	1.282
rcm_magnitude_rcm_signed_magnitude_64	64.29	67.98	66.25	63.50	85.55	94.53	4.02	1.218
rcm_magnitude_rcm_signed_magnitude_128	64.60	67.94	66.77	64.30	86.79	94.32	4.02	1.048
rcm_magnitude_symmetry_64	63.67	64.68	65.37	62.66	86.17	94.43	3.20	2.765
rcm_magnitude_symmetry_128	65.02	66.05	66.83	64.23	87.20	94.84	2.99	3.063
rcm_signed_magnitude_log_density_64	63.47	66.25	65.69	62.40	85.66	93.60	4.23	1.057
rcm_signed_magnitude_log_density_128	63.36	66.43	65.51	62.54	86.27	94.53	4.13	1.228
rcm_signed_magnitude_rcm_log_density_64	64.40	66.05	65.84	63.42	85.96	94.12	3.92	1.072
rcm_signed_magnitude_rcm_log_density_128	63.67	66.50	65.11	62.52	85.86	94.01	4.02	1.263
symmetry_log_density_64	64.50	66.70	66.88	63.57	87.10	95.15	3.10	2.540
symmetry_log_density_128	65.12	66.68	67.14	64.10	86.79	94.84	3.10	3.336
symmetry_rcm_log_density_64	64.29	67.12	66.69	63.43	85.66	94.01	3.10	4.058
symmetry_rcm_log_density_128	64.71	66.92	66.98	63.84	86.58	95.36	2.99	2.126
symmetry_rcm_signed_magnitude_64	64.09	65.96	65.58	62.62	84.31	93.29	3.20	1.817
symmetry_rcm_signed_magnitude_128	64.09	65.31	66.05	62.89	86.69	95.15	3.10	2.710

The dual-channel experiments are summarised in Table 5.3 (Campaign 3, 14 features, 64px) and Table 5.4 (Campaign 4, 20 features, 64px and 128px). For Campaign 3 the six best image modes from Campaign 1 - magnitude, rcm_signed_magnitude, rcm_magnitude, signed_magnitude, symmetry, and density - together with sign are combined into all 21 pairwise combinations. For Campaign 4 only the six best image modes from Campaign 2 - magnitude, rcm_magnitude, rcm_log_density, symmetry, log_density, and rcm_signed_magnitude - are combined, yielding 15 pairs each evaluated at two resolutions (64px and 128px).

The dominant finding over these two Campaigns is that magnitude is the most accurate channel also here. Every top-performing pair across both campaigns includes

it as one channel. Replacing magnitude with any other mode as the primary channel reduces accuracy by 2–3 pp.

The best second channel is `log_density` (best F1) or `rcm_log_density` (best accuracy), both outperforming `symmetry` and `rcm_signed_magnitude`. The combination `rcm_log_density+log_density` (two log-family channels) performs worst in Campaign 4, confirming that redundant modes do not benefit the model.

128 px consistently outperforms 64 px by +0.5–+1.3 pp across all mode pairs, at a cost of roughly 4× longer training time and 4× higher inference latency. The best Campaign 4 single model is `magnitude+rcm_log_density` at 128 px with 66.56% accuracy and 65.53% F1.

Dual-channel models outperform the best single-channel model by +2.3 pp in accuracy (66.56% vs. 64.29%), confirming that the two image channels carry complementary discriminative information beyond what either channel provides alone.

5.5 Effect of Image Resolution

In the dual-channel experiment with 64 and 128 px images, shown in Table 5.4, the increase in resolution brings a consistent improvement of 0.5–1.3 pp in accuracy and F1 values across all 15 mode pairs. The largest gain is seen in `magnitude+rcm_log_density`, where accuracy improves by 1.13 pp and F1 by 0.79 pp; the smallest is on `magnitude+log_density`, where accuracy stays the same and only F1 increases by 0.58 pp. No pair degrades at higher resolution.

The cost of the higher resolution is an 4× increase in inference time, as the dual-channel 64 px model runs at approximately 2.8 ms/sample and the 128 px model takes about 10.9 ms/sample. The training time per model is around 13 min at 64 px and around 48 min at 128 px, on a NVIDIA GeForce RTX 5060 Laptop GPU.

In the single-channel experiments, results were more mixed: `binary_128` and `density_128` outperformed their 64 px counterparts, but `magnitude_64` beat `magnitude_128` in accuracy and F1. This suggests that 64 px is already sufficient for magnitude-encoded images, where per-cell absolute values matter more than the fine spatial arrangement of non-zeros.

5.6 Ensemble vs. Single Best Model

Table 5.5: Campaign 5 - Ensemble results: softmax-averaging of four independently trained dual-channel models. *Campaign 3*: 14 features, 64px, no penalty; members are mag+signed_mag, mag+rcm_signed_mag, mag+symmetry, rcm_mag+signed_mag. *Campaign 4*: 20 features, 128px, $\lambda = 1.0$; members are mag+log_density, mag+rcm_log_density, rcm_mag+rcm_signed_mag, symmetry+log_density. Bold marks the column best.

Ensemble	Acc%	MP%	MR%	F1%	Top-2%	Top-3%	Fail%	MRT×
Campaign 3 ensemble (64px, 4 models)	67.39	69.47	69.58	66.37	87.82	94.94	2.79	4.055
Campaign 4 ensemble (128px, 4 models)	67.29	70.37	69.48	66.77	87.41	94.94	3.92	1.043

Campaign 3 ensemble.

The Campaign 3 ensemble — magnitude+signed_magnitude, magnitude+rcm_signed_magnitude, magnitude+symmetry, and rcm_magnitude+signed_magnitude (all 64px) — achieves 67.39% accuracy and an F1 of 66.37%, surpassing the best individual member (magnitude+signed_magnitude at 66.05% accuracy and 65.32% F1) by +1.34pp in accuracy and +1.05pp in F1. The failure rate drops to 2.79%, below all four member models individually, confirming that non-converging predictions are not correlated across members.

Campaign 4 ensemble. The Campaign 4 ensemble — magnitude+log_density, magnitude+rcm_log_density, rcm_magnitude+rcm_signed_magnitude, and symmetry+log_density (all 128px) — achieves 67.29% accuracy and 66.77% F1, compared to the best individual member (magnitude+rcm_log_density_128 at 66.56% accuracy and 65.53% F1), a gain of +0.73pp in accuracy and +1.24pp in F1. The failure rate of 3.92% and mean runtime ratio of 1.043× remain below or comparable to all individual members.

Notably, the Campaign 4 ensemble surpasses the Campaign 3 ensemble only in F1 (66.77% vs. 66.37%), while trailing in accuracy (67.29% vs. 67.39%). This comes at notably higher inference cost, running at approximately 46.5ms/sample — roughly 4× slower than the Campaign 3 ensemble (≈ 12 ms) — due to the 128px images.

Inference cost. The ensemble multiplies inference time by the number of members (4×). At ≈ 12 ms/sample for four dual-channel 64px models (Campaign 3) and ≈ 46.5 ms/sample for four dual-channel 128px models (Campaign 4), the prediction overhead remains negligible compared to typical iterative solver runtimes which range from tens of milliseconds to several seconds per solve.

5.7 Comparison to MM-AutoSolver

This section compares the thesis model against the MM-AutoSolver paper results and the replication performed as a baseline. The replication was trained on a synthetic dataset with some real SuiteSparse matrices included, but was evaluated on the same 19 solver classes with slightly different per-class distributions. The replication even exceeds the paper in accuracy by 3.43 pp, as seen in Table 5.6, and may be the result of the slightly different data source (Section 3.2).

Table 5.6: Comparison against MM-AutoSolver [Xio+25]. Best single: magnitude+rcm_log_density at 128 px (Campaign 4). Ensemble C3: 4 models, 64 px, 14 features. Ensemble C4: 4 models, 128 px, 20 features, $\lambda = 1.0$. Bold marks the best thesis result in each column.

System	Acc%	MP%	MR%	F1%
MM-AutoSolver (paper)	78.54	63.41	62.81	62.53
MM-AutoSolver (replication)	81.97	62.88	61.83	60.45
This thesis (best single, C4)	66.56	69.15	68.88	65.53
This thesis (ensemble C3, 64 px)	67.39	69.47	69.58	66.37
This thesis (ensemble C4, 128 px)	67.29	70.37	69.48	66.77

A direct comparison between the MM-AutoSolver approach and the thesis approach is misleading for two reasons. First, the class distribution in the dataset differs substantially from the MM-AutoSolver dataset, which uses a dataset where the fbcgsr+jacobi and bcgs1+none classes make up more than 36% of the whole distribution. A model can exploit this by defaulting to one of these dominant classes when uncertain, inflating accuracy. The thesis dataset was constructed with a minimal per-class constraint of 250 matrices and capped at 600 matrices per class, resulting in a much flatter distribution and harder problem. Second, the training matrices also differ, as the MM-AutoSolver paper uses more real-world application matrices generated within FreeFEM and OpenFOAM. Within the thesis, only SuiteSparse matrices with real-world applications are used; the remainder consists of synthetically generated ones.

Despite the lower accuracy, the macro-averaged metrics are less sensitive to class distribution and can still be compared. The thesis model even achieves around 7 pp better results in Macro Precision and Macro Recall, meaning that despite the harder problem it learned to distinguish the classes better than the MM-AutoSolver approach.

Another insight is the Top-3 accuracy of around 94%, meaning that even when the predicted solver is not the best, in nearly all cases it is among the top three choices.

Combined with an average failure rate of around 3–4%, the approach is practically viable.

5.8 Per-Solver Analysis

Table 5.7 breaks down per-class F1, precision, recall, and failure rate for the Campaign 4 ensemble (members: magnitude+log_density, magnitude+rcm_log_density, rcm_magnitude+rcm_signed_magnitude, symmetry+log_density; 20 features, 128px, $\lambda = 1.0$; 67.29% Acc, 66.77% F1). The 19 classes span a wide performance range, from minres+gamg at 88.7% F1 down to gmres+gamg at 20.9% F1, illustrating that solver prediction difficulty varies considerably across the label space.

Table 5.7: Per-class results for the Campaign 4 ensemble (4 models: magnitude+log_density, magnitude+rcm_log_density, rcm_magnitude+rcm_signed_magnitude, symmetry+log_density; 128px, $\lambda = 1.0$). Sorted by F1 descending. N is the number of training samples per class.

Solver pair	N	F1%	Prec%	Rec%	Fail%
minres+gamg	600	88.7	80.8	98.3	0.0
cr+eisenstat	600	88.3	96.1	81.7	0.0
symmlq+sor	250	86.3	84.6	88.0	15.4
symmlq+jacobi	253	82.1	74.2	92.0	3.2
bcgsl+asm	428	73.4	78.4	69.0	2.7
fbcgsl+jacobi	600	72.3	72.9	71.7	1.7
cr+ilu	600	72.1	71.0	73.3	6.5
cr+jacobi	600	71.9	75.9	68.3	0.0
cg+bjacobi	600	71.9	67.6	76.7	4.4
cg+eisenstat	600	70.7	73.2	68.3	0.0
bcgsl+none	600	70.0	70.0	70.0	0.0
dgmres+none	354	66.7	64.9	68.6	0.0
symmlq+icc	600	63.8	56.4	73.3	5.1
fbcgsl+ilu	600	63.4	78.0	53.3	0.0
cgs+gamg	336	52.8	38.4	84.8	0.0
fcg+gamg	290	52.3	39.0	79.3	3.4
cg+ilu	600	52.1	52.5	51.7	30.5
fgmres+gamg	600	49.0	63.2	40.0	0.0
gmres+gamg	600	20.9	100.0	11.7	0.0

Easy classes. `minres+gamg` (88.7% F1, recall 98.3%) leads the ranking, reliably identifying symmetric indefinite problems that are unsuitable for CG-type solvers. `cr+eisenstat` (88.3% F1, precision 96.1%) follows closely, dominating structurally symmetric and nearly diagonally dominant matrices. `symmlq+jacobi` (82.1% F1, recall 92.0%) shows high recall but lower precision (74.2%), indicating the model confidently identifies this class but occasionally predicts it for matrices belonging to neighbouring symmetric solver classes.

Hard classes. `gmres+gamg` is the hardest class (20.9% F1) with a recall of only 11.7% despite perfect precision of 100.0%: the model is always right when it predicts this class, but almost never does so. This is a consequence of class ambiguity within the GAMG-preconditioned family: `fgmres+gamg` (49.0% F1) and `fcg+gamg` (52.3% F1) converge in similar time on many of the same matrices, and the model spreads probability across the family rather than committing to one member. `fbcgsl+ilu` (63.4% F1, recall 53.3%) and `symmlq+icc` (63.8% F1) are the next weakest: both are specialised combinations whose winning conditions are narrow and hard to separate from neighbouring classes by image alone.

Failure rates. `cg+ilu` stands out with a 30.5% failure rate. This is likely a side-effect of the convergence penalty interacting with the new ILU-related features (`min_diag_dominance`, `diag_dom_variance`): the penalty reduces failure rates globally but appears to shift prediction errors for borderline cases where the new features signal marginal ILU suitability. `symmlq+sor` (15.4%) and `cr+ilu` (6.5%) also carry elevated failure rates, as both rely on preconditioners sensitive to spectral properties not fully captured by the feature set. Ten classes achieve 0% failure rate: `minres+gamg`, `cr+eisenstat`, `gmres+gamg`, `cr+jacobi`, `cg+eisenstat`, `bcgsl+none`, `fbcgsl+ilu`, `dgmres+none`, `cgs+gamg`, and `fgmres+gamg`.

5.9 Practical Inference Cost

Model inference is fast enough to be neglectable compared to actual solver runtimes. Table 5.8 summarises the per-sample cpu latency for the main model configurations, measured on the validation set of 969 samples in the evaluation set.

Table 5.8: CPU inference latency per sample (averaged over 969 validation samples).

Configuration	Latency (ms/sample)	Model params
MLP only (nocnn)	≈ 0.08	≈ 57 K
Single-channel CNN, small model (64 px)	≈ 1.2	≈ 0.5 M
Single-channel CNN, small model (128 px)	≈ 4.5	≈ 0.5 M
Single-channel CNN, large model (64 px)	≈ 2.8	≈ 2.1 M
Dual-channel CNN, large model (64 px)	≈ 3.0	≈ 2.1 M
Dual-channel CNN, large model (128 px)	≈ 10.9	≈ 2.1 M
Campaign 3 ensemble (64 px, 4 models)	≈ 12.0	4×2.1 M
Campaign 4 ensemble (128 px, 4 models)	≈ 46.5	4×2.1 M

6 Conclusion

6.1 Summary of Findings

6.2 Contributions

6.3 Limitations

6.4 Future Work

CG Conjugate Gradient

GMRES Generalized Minimal Residual

FGMRES Flexible Generalized Minimal Residual

List of Figures

- 3.1 Distribution of matrix sizes (number of rows n) for the main thesis dataset (9,711 matrices, Campaigns 1–4). The x -axis uses a logarithmic scale. 11
- 4.1 Distribution of matrix sizes (number of rows n) for the MM-AutoSolver replication dataset (23,167 matrices). The x -axis uses a logarithmic scale. 22

List of Tables

3.1	Synthetic generator buckets grouped by structural matrix property. Size variants of the same generator function are merged into one row. Solver requirements follow [Saa03]: CG requires SPD (Ch. 6), MINRES/SYMMMLQ require symmetry (Ch. 6), GMRES and BiCGS variants apply to nonsymmetric systems (Ch. 7). Primary targets are theoretically motivated and empirically confirmed.	8
3.2	Class distribution in the full dataset (9711 samples). The label of each matrix is the fastest converging solver-preconditioner pair. Classes are capped at 600.	10
3.3	17 numerical features used in MM-AutoSolver [Xio+25] (Table 2). . . .	12
3.4	Original 14 matrix features used in the single-channel experiments. . .	13
3.5	The 6 additional matrix features introduced in Campaign 2, expanding the feature vector from 14 to 20 entries.	14
3.6	The 14 image modes evaluated in the single-channel experiments. The six <code>rcm_*</code> variants apply Reverse Cuthill–McKee reordering to the matrix before rendering with the corresponding base mode.	15
4.1	Per-class sample counts: MM-AutoSolver paper dataset vs. replication dataset used in this thesis.	21
5.1	Campaign 1 — Single-channel: 14 features, no convergence penalty, small model.	26
5.2	Campaign 2 — Single-channel: 20 features, convergence penalty $\lambda = 1.0$, large model, 64 px.	28
5.3	Campaign 3 — Dual-channel: 14 features, no convergence penalty, large model, 64 px.	30
5.4	Campaign 4 — Dual-channel: 20 features, convergence penalty $\lambda = 1.0$, large model, 64 px and 128 px.	31

5.5	Campaign 5 - Ensemble results: softmax-averaging of four independently trained dual-channel models. <i>Campaign 3</i> : 14 features, 64 px, no penalty; members are mag+signed_mag, mag+rcm_signed_mag, mag+symmetry, rcm_mag+signed_mag. <i>Campaign 4</i> : 20 features, 128 px, $\lambda = 1.0$; members are mag+log_density, mag+rcm_log_density, rcm_mag+rcm_signed_mag, symmetry+log_density. Bold marks the column best.	33
5.6	Comparison against MM-AutoSolver [Xio+25]. Best single: magnitude+rcm_log_density at 128 px (Campaign 4). Ensemble C3: 4 models, 64 px, 14 features. Ensemble C4: 4 models, 128 px, 20 features, $\lambda = 1.0$. Bold marks the best thesis result in each column.	34
5.7	Per-class results for the Campaign 4 ensemble (4 models: magnitude+log_density, magnitude+rcm_log_density, rcm_magnitude+rcm_signed_magnitude, symmetry+log_density; 128 px, $\lambda = 1.0$). Sorted by F1 descending. N is the number of training samples per class.	35
5.8	CPU inference latency per sample (averaged over 969 validation samples).	37

Bibliography

- [BAM18] T. Baltrušaitis, C. Ahuja, and L.-P. Morency. “Multimodal machine learning: A survey and taxonomy.” In: *IEEE transactions on pattern analysis and machine intelligence* 41.2 (2018), pp. 423–443.
- [CM69] E. Cuthill and J. McKee. “Reducing the bandwidth of sparse symmetric matrices.” In: *Proceedings of the 1969 24th national conference*. 1969, pp. 157–172.
- [LH19] I. Loshchilov and F. Hutter. “Decoupled Weight Decay Regularization.” In: *International Conference on Learning Representations (ICLR)* (2019).
- [MV77] J. A. Meijerink and H. A. Van Der Vorst. “An iterative solution method for linear systems of which the coefficient matrix is a symmetric M-matrix.” In: *Mathematics of Computation* 31.137 (1977), pp. 148–162.
- [Saa03] Y. Saad. *Iterative methods for sparse linear systems*. SIAM, 2003.
- [Stü01] K. Stüben. “A review of algebraic multigrid.” In: *Numerical Analysis: Historical Developments in the 20th Century* (2001), pp. 331–359.
- [Xio+25] S. Xiong et al. “MM-AutoSolver: A multimodal machine learning method for the auto-selection of iterative solvers and preconditioners.” In: *Journal of Parallel and Distributed Computing* 205 (2025), p. 105144.